

Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Задание по курсу «Суперкомпьютерное моделирование и технологии»

Задание 2, Вариант 5

Выполнил:
студент м221 группы
Басалов Ярослав Александрович

Москва, 2026

Содержание

Введение **3**

1 Математическая постановка дифференциальной задачи **4**

2 Численный метод решения задачи **5**

3 Вариант задания **6**

4 Программная реализация **7**

5 Исследование масштабируемости **10**
5.1 Результаты расчетов для OpenMP программы 10

Заключение **14**

Список литературы **15**

Введение

В качестве модельной задачи предлагается задача для трехмерного гиперболического уравнения в области, представляющей из себя прямоугольный параллелепипед. Индивидуальные варианты заданий отличаются типом граничных условий. Данное уравнение часто применяется в теории тепло и массопереноса, гидро и аэромеханике, электростатике. Поэтому поиск решения данный задачи в различных областях является весьма актуальным.

Задание необходимо выполнить на следующих ПВС Московского университета:

1. IBM Polus [1].

1 Математическая постановка дифференциальной задачи

Рассматривается трёхмерная замкнутая область

$$\Omega = [0 \le x \le L_x] \times [0 \le y \le L_y] \times [0 \le z \le L_z]$$

для $(0 < t \le T]$ требуется найти решение $u(x, y, z, t)$ уравнения в частных производных

$$\frac{\partial^2 u}{\partial t^2} = \Delta u \tag{1}$$

с начальными условиями

$$u\big|_{t=0} = \varphi(x, y, z), \tag{2}$$

$$\left.\frac{\partial u}{\partial t}\right|_{t=0} = 0, \tag{3}$$

при условии, что на границах области заданы однородные граничные условия первого рода

$$u(0, y, z, t) = 0, \qquad u(L_x, y, z, t) = 0, \tag{4}$$

$$u(x, 0, z, t) = 0, \qquad u(x, L_y, z, t) = 0, \tag{5}$$

$$u(x, y, 0, t) = 0, \qquad u(x, y, L_z, t) = 0, \tag{6}$$

либо периодические граничные условия

$$u(0, y, z, t) = u(L_x, y, z, t), \qquad u_x(0, y, z, t) = u_x(L_x, y, z, t), \tag{7}$$

$$u(x, 0, z, t) = u(x, L_y, z, t), \qquad u_y(x, 0, z, t) = u_y(x, L_y, z, t), \tag{8}$$

$$u(x, y, 0, t) = u(x, y, L_z, t), \qquad u_z(x, y, 0, t) = u_z(x, y, L_z, t). \tag{9}$$

Конкретная комбинация граничных условий определяется индивидуальным вариантом задания (см. п. 3)

2 Численный метод решения задачи

Содержание данного пункта основано на материале книги [2].

Для численного решения задачи введем на Ω сетку $\omega_{h\tau} = \overline{\omega}_h \times \omega_\tau$, где

$$T = T_0,$$
$$L_x = L_{x0}, L_y = L_{y0}, L_z = L_{z0}$$
$$\overline{\omega}_h = \{\left(x_i = ih_x, y_j = jh_y, z_k = kh_z\right), i, j, k = 0, 1, \ldots, N, h_x N = L_x, h_y N = L_y, h_z N = L_z\},$$
$$\omega_\tau = \{t_n = n\tau, n = 0, 1, \ldots, K, \tau K = T\}.$$

Через ω_h обозначим множество внутренних, а через γ_h множество граничных узлов сетки $\overline{\omega}_h$.

Для аппроксимации исходного уравнения (1) с однородными граничными условиями (4)–(6) и начальными условиями (2)–(3) воспользуемся следующей системой уравнений:

$$\frac{u^{n+1}_{ijk} - 2u^n_{ijk} + u^{n-1}_{ijk}}{\tau^2} = \Delta_h u^n, \qquad (x_i, y_j, z_k) \in \omega_h, \qquad n = 1, 2, \ldots, K-1,$$

Здесь Δ_h — семиточечный разностный аналог оператора Лапласа:

$$\Delta_h u^n = \frac{u^n_{i-1,j,k} - 2u^n_{i,j,k} + u^n_{i+1,j,k}}{h^2} + \frac{u^n_{i,j-1,k} - 2u^n_{i,j,k} + u^n_{i,j+1,k}}{h^2} + \frac{u^n_{i,j,k-1} - 2u^n_{i,j,k} + u^n_{i,j,k+1}}{h^2}.$$

Приведенная выше разностная схема является явной – значения u^{n+1}_{ijk} на $(n+1)$-м шаге можно явным образом выразить через значения на предыдущих слоях.

Для начала счета (т.е. для нахождения u^2_{ijk}) должны быть заданы значения u^0_{ijk}, u^1_{ijk}, $(x_i, y_j, z_k) \in \omega_h$. Из условия (2) имеем

$$u^0_{ijk} = \varphi(x_i, y_j, z_k), \qquad (x_i, y_j, z_k) \in \omega_h. \tag{10}$$

Простейшая замена начального условия (3) уравнением $(u^1_{ijk} - u^0_{ijk})/\tau = 0$ имеет лишь первый порядок аппроксимации по τ. Аппроксимацию второго порядка по τ и h дает разностное уравнение

$$\frac{u^1_{ijk} - u^0_{ijk}}{\tau} = \frac{\tau}{2}\Delta_h \varphi(x_i, y_j, z_k), \qquad (x_i, y_j, z_k) \in \omega_h. \tag{11}$$

$$u^1_{ijk} = u^0_{ijk} + \frac{\tau^2}{2}\Delta_h \varphi(x_i, y_j, z_k) \tag{12}$$

Разностная аппроксимация для периодических граничных условий выглядит следующим образом

$$u^{n+1}_{0jk} = u^{n+1}_{Njk}, \qquad u^{n+1}_{1jk} = u^{n+1}_{N+1jk},$$
$$u^{n+1}_{i0k} = u^{n+1}_{iNk}, \qquad u^{n+1}_{i1k} = u^{n+1}_{iN+1k},$$
$$u^{n+1}_{ij0} = u^{n+1}_{ijN}, \qquad u^{n+1}_{ij1} = u^{n+1}_{ijN+1},$$

$i, j, k = 0, 1, \ldots, N$.

Для вычисления значений $u^0, u^1, \in \gamma_h$ допускается использование аналитического значения u, которое задается в программе еще для вычисления погрешности решения задачи.

3 Вариант задания

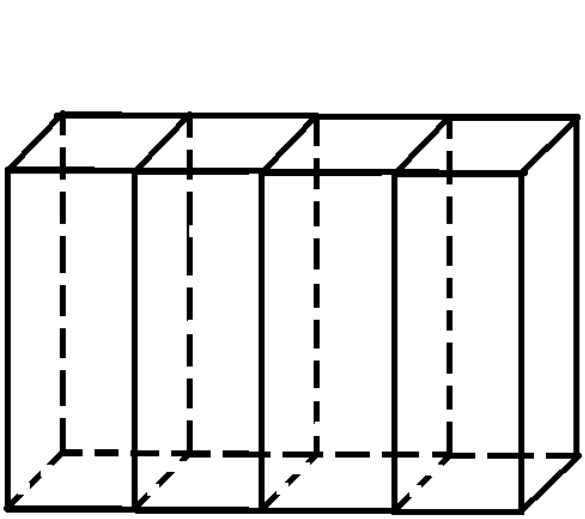
Индивидуальные варианты заданий отличаются комбинацией граничных условий. В текущей работе рассматривается вариант 5:

- $u_{analytical} = sin\left(\frac{2\pi}{L_x}x\right) \cdot sin\left(\frac{\pi}{L_y}y\right) \cdot sin\left(\frac{\pi}{L_z}z\right) \cdot cos\left(a_t \cdot t + 2\pi\right), a_t = \pi\sqrt{\frac{4}{L_x^2} + \frac{1}{L_y^2} + \frac{1}{L_z^2}},$
- x – периодическое условие (7),
- y – условие первого рода (5),
- z – условие первого рода (6).

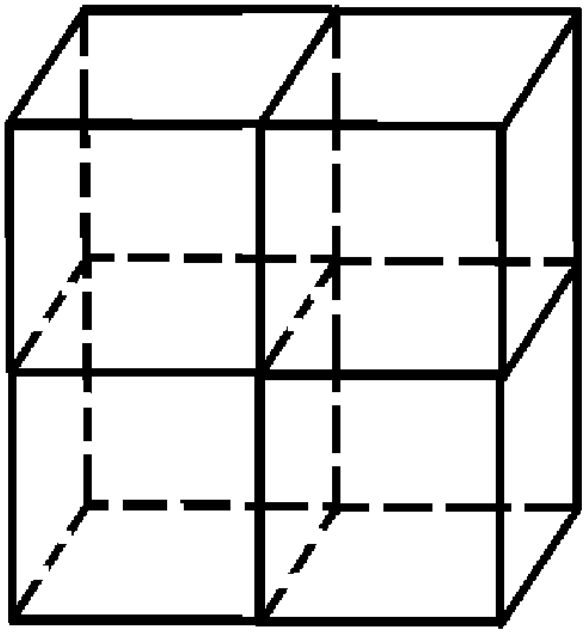
Функция $u_{analytical}$ напрямую зависит от введенных L_x, L_y, L_z, которые являются входными аргументами программы. Функция $\varphi(x, y, z)$ зависит напрямую от $u_{analytical}$ (2).

4 Программная реализация

Требуется реализовать блочное разбиение области между процессами, поскольку в этом случае предполагается меньше межпроцессорных коммуникаций, по сравнению с ленточным.



Ленточное разбиение 3d



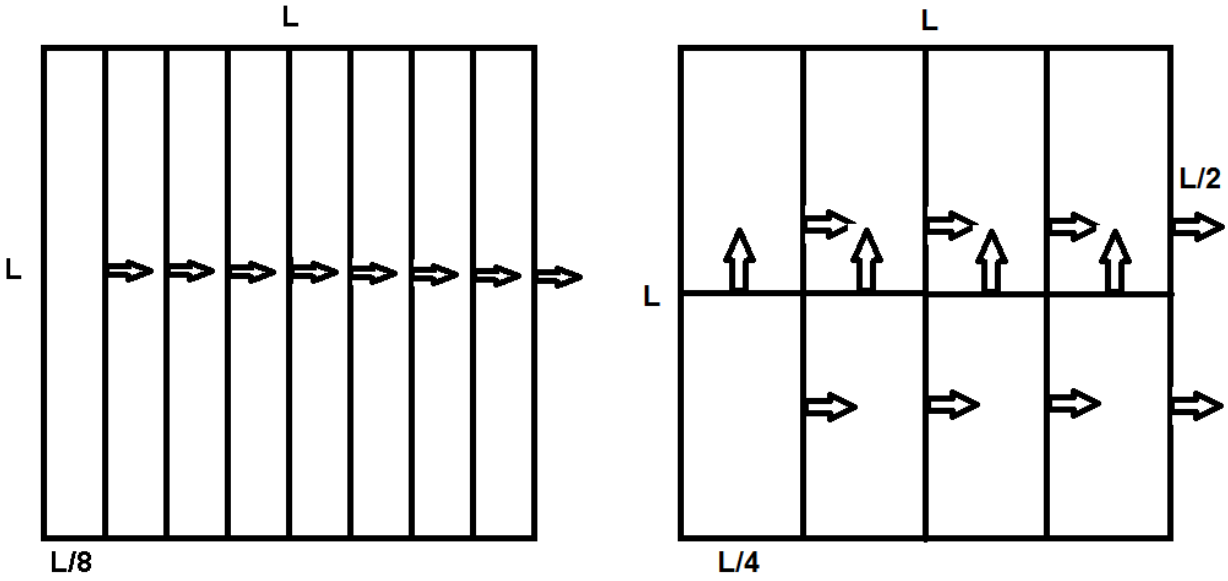
Блочное разбиение 3d

Рис. 1: Виды разбиений 3d

Число областей, на которое необходимо произвести разбиение может быть произвольным, как четным, так и нечетным.

На рисунке 1 изображен схематично двумерный ленточный и блочный вариант разбиения квадратной области на 8 частей.

В случае ленточного разбиения получаем $S_{strip} = 8L$ межпроцессорных коммуникаций. В блочном случае $S_{block} = 8(L/2) + 4(L/4) = 5L$ коммуникаций, то есть в 1.6 раз меньше. С ростом числа разбиений (и процессов, соответственно) это число будет только увеличиваться. Данные рассуждения можно обобщить и на трехмерный случай.



Ленточное разбиение Блочное разбиение

Рис. 2: Виды разбиений

Таким образом, алгоритм решения задачи, выглядит следующим образом:

1. Исходя из варианта, расчитывается точное аналитическое решение $u_{analytical}$ в узлах сетки.
2. Проводим разбиение области Ω между процессами.
3. Фиксируем временной слой (начиная с 0).
4. Используя формулы (10) и (12). находим значения u^0 и u^1.
5. Пользуясь найденными u^0 и u^1 и разностным представлением уравнения (1), находим значения u в локальной области разбиения.
6. Передаем посчитанные граничные значения блокам-соседям.
7. Повторяем шаги 5–6 для внутренних блоков.
8. Определяем максимальную погрешность на сетке между посчитанным и аналитическим решением.
9. Переходим на следующий слой по времени и повторяем шаги 2–9.

Реализация данного алгоритма была написана на языке C++ и выложена на GitHub (https://github.com/Marilius/edu-cmc-skmodel26). Для компиляции и запуска использовались подобные команды (версия с OpenMP):

Листинг 1: Пример запуска с OpenMP

```
g++ -O2 -fopenmp -std=c++11 src/main.cpp -o solver.out
mpisubmit.pl -t 8 solver.out --stdout std.out --stderr std.err \
-- 128 20 1. 1. 1. 0.01
```

5 Исследование масштабируемости

5.1 Результаты расчетов для OpenMP программы

Число OpenMP-нитей / послед. алгоритм	Размер сетки	Время решения	Ускорение
serial	128^3	12.38718873	1.0
1	128^3	11.906204419	1.0403977870758243
2	128^3	6.179221872	2.0046518779541245
4	128^3	3.195486474	3.876464141152863
8	128^3	2.020456917	6.1308848636043445
serial	256^3	98.77836788	1.0
1	256^3	94.707646745	1.0429819689846207
2	256^3	48.758578746	2.0258664304915466
4	256^3	25.047241724	3.943682460865606
8	256^3	15.251786104	6.476511485700285
serial	512^3	787.55294798	1.0
1	512^3	754.75414806	1.043456269838735
2	512^3	384.922989328	2.0460013296553496
4	512^3	199.994196498	3.9378790073434744
5	512^3	122.05060391	6.4526755521893255

Таблица 1: Время решения, $L_x = L_y = L_z = 1$

Последовательный и параллельный варианты программы были скомпилированы с флагом оптимизации -O2. При любом числе потоков получили проигрыш по сравнению с последовательным вариантом алгоритма. Компилятор GCC довольно хорошо оптимизирует последовательный код.

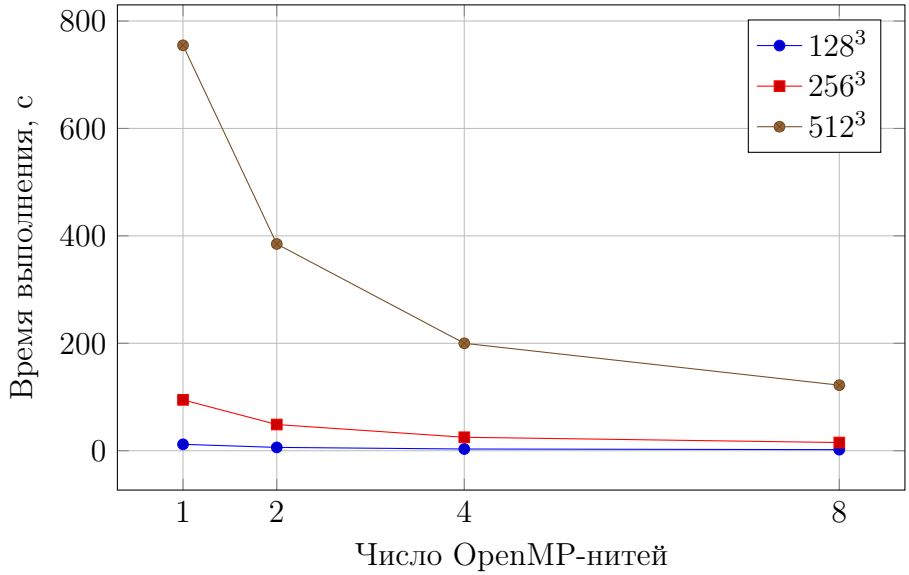


Рис. 3: Время выполнения OpenMP-реализации при различных размерах сетки $L_x = L_y = L_z = 1$

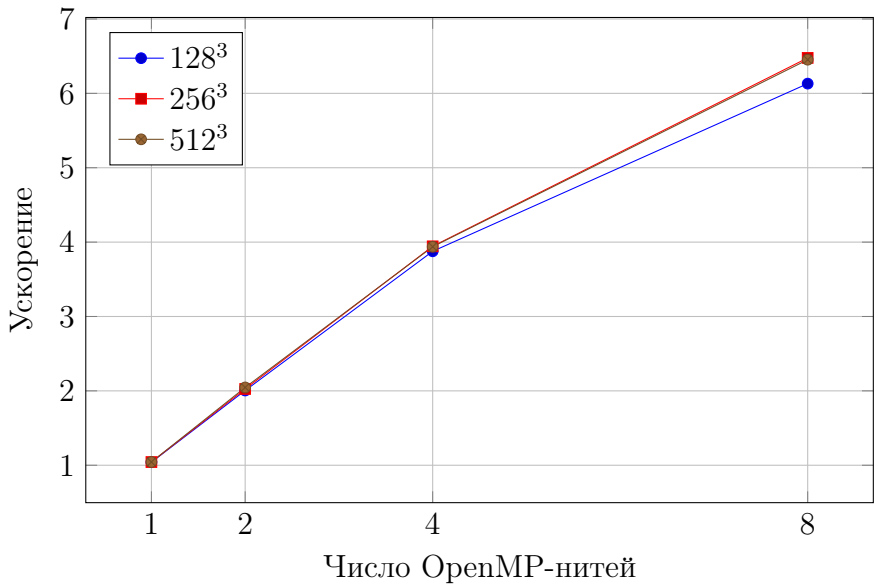


Рис. 4: Ускорение OpenMP-реализации при различных размерах сетки $L_x = L_y = L_z = 1$

Число OpenMP-нитей / послед. алгоритм	Размер сетки	Время решения	Ускорение
serial	128^3	12.371096542	1.0
1	128^3	12.418100815	0.9962148581574387
2	128^3	8.258355857	1.4980096227645523
4	128^3	4.719976269	2.6210082078698687
8	128^3	2.868507856	4.312728834304437
serial	256^3	98.502743678	1.0
1	256^3	98.622051571	0.998790251357587
2	256^3	53.337296491	1.8467892105221553
4	256^3	32.507485622	3.030155725465786
8	256^3	15.746087913	6.255696286102655
serial	512^3	785.029544667	1.0
1	512^3	753.413243477	1.0419640900445164
2	512^3	399.393005665	1.9655565659190875
4	512^3	227.560406705	3.4497633223370014
5	512^3	139.734042225	5.618026446289616

Таблица 2: Время решения, $L_x = L_y = L_z = \pi$

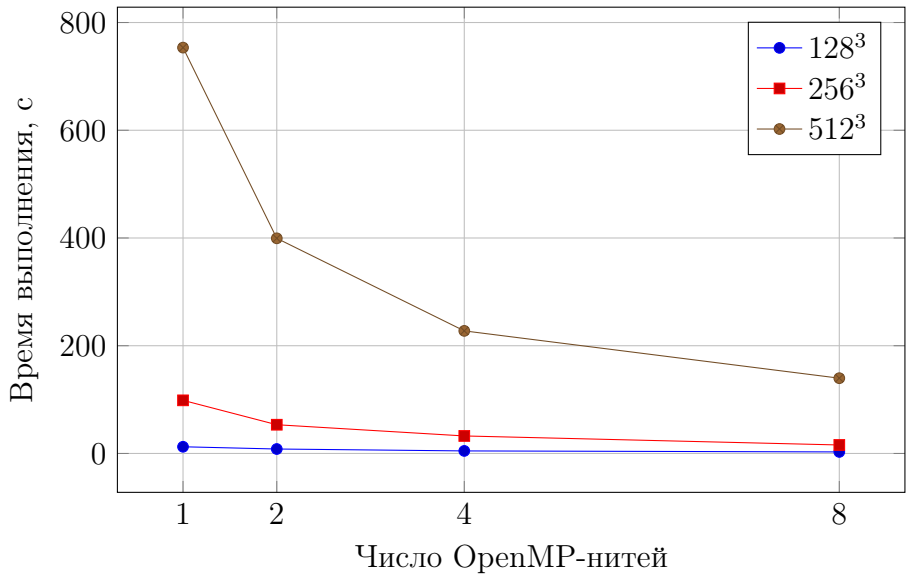


Рис. 5: Время выполнения OpenMP-реализации при различных размерах сетки $L_x = L_y = L_z = \pi$

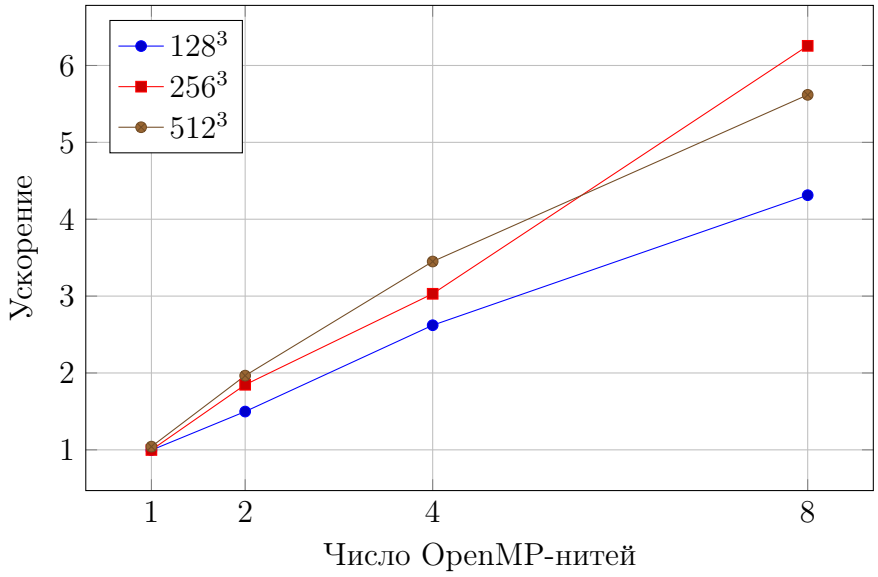


Рис. 6: Ускорение OpenMP-реализации при различных размерах сетки $L_x = L_y = L_z = \pi$

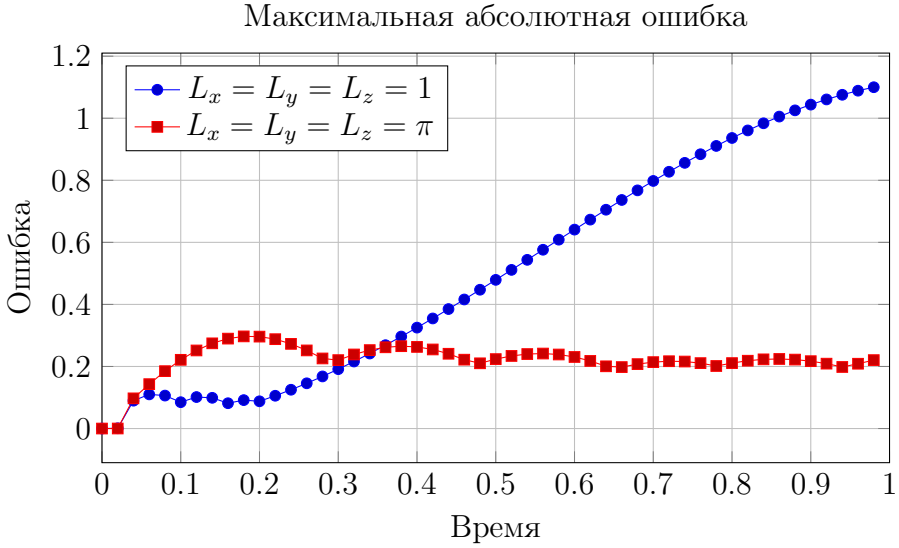


Рис. 7: Размер сетки = 128^3, 50 шагов по времени

Заключение

TODO

Список литературы

[1] Суперкомпьютер IBM Polus. `http://hpc.cs.msu.su/polus`.

[2] Численные методы / Гулин А.В. Самарский А.А. // *М.: Наука. Гл. ред. физ-мат. лит.* — 1989.